

The NULL Merchant and Other Silent Precondition Traps for Data-Science Agents

An exploratory benchmark-development study of coverage-validation omission in ten agent configurations

Alexander J. King

Independent researcher, alexander.julian.king@gmail.com

July 2026

Abstract

Coding agents verify the data they are given far more deeply than single-turn models suggest: in development experiments, traps built on misleading documentation, false task premises, and known statistical pitfalls with named defenses were almost uniformly caught by frontier agents. We identify a family that survives: **instrument-precondition traps**, in which the agent’s habitual tool template (a field-position parser, a CSV reader’s defaults) carries a precondition that is silently violated in a region of the data that routine spot-checks never touch, yielding a plausible wrong answer. On a frozen 10-task battery ($n=5$ per cell; 500 runs), ten agent configurations spanning eight OpenAI and Anthropic models fell (reported the exact planted wrong answer) at rates from 26% to 100%, every OpenAI-family configuration above every Anthropic one; 329/330 answered-but-wrong runs (99.7%) were the *planted answer*. The three GPT-5.6 models, released after this first battery froze, fell at 82–90% on it prospectively, with zero out-of-scope file access in transcript audits. A one-paragraph hygiene preamble reduced fall rates about fourfold and generalized across trap classes, but left model-specific residues: one task (a merchant whose short-code is literally NULL, silently swallowed by pandas’ default `na_values`) fell in 49/50 unwarned runs across all configurations and resisted the preamble for one configuration. The battery’s traps were designed by an Anthropic-lineage model; a second frozen battery, designed by GPT-5.6 Sol under the same frame and a pre-registered protocol, replicated the family ordering (pooled falls 98% vs. 79%) while narrowing the gap, and reproduced the planted-answer specificity (99.1%). All generators, environments, transcripts, run ledgers, and failed trap designs are released.

1 Introduction

SimpleBench [1] demonstrated a durable gap between non-specialized humans (83.7%) and frontier LLMs ($\leq 41.7\%$ at its publication) using questions that *look* like familiar templates (an averages word problem, a classic riddle) while an everyday physical or social detail invalidates the template. A substantial literature locates the mechanism in the statistics of pretraining: models retrieve high-frequency solution patterns and execute them even when a stated detail invalidates the pattern [2, 3, 4, 5, 6, 7, 8], with failures that are systematic rather than random: models converge on the *same* wrong answer, the one the misdirection was designed to elicit [1].

Modern coding agents complicate this picture. Given tools, they inspect the data before answering: they read documentation, profile distributions, check nulls and ranges, and challenge task premises. In pilot experiments (§4), every trap we transplanted from the single-turn literature

(misleading documentation, false bug reports, leaky features, statistical pitfalls) was caught by frontier agents, often with verification exceeding our answer keys. The single-turn failure mechanism does not transfer directly, because agentic verification is genuinely deep *where the agent looks*.

This study asks: **where do agents not look?** The answer, developed iteratively and then tested on a frozen battery (locked before any scored run, and unchanged after) across ten configurations, is: *at their own instruments*. An agent that reads every file and profiles every column will still write a field-position parser and never check whether every line parsed; we call this *coverage-validation omission*: skipping the check that one’s instrument actually covered every record in scope. In one documented run, an agent produced an hourly error table containing a multi-hour hole in a production service’s log and reported it, uninvestigated, as “Hours 10:00–13:00 logged zero errors.” This is the SimpleBench failure transposed from facts to procedures: a high-frequency template retrieved and executed past the detail that invalidates it, with errors converging on the designed answer rather than scattering. We characterize the conditions under which such traps succeed, measure fall rates at $n=5$ across ten agent configurations from two labs, test elicitation and effort explanations experimentally, report that the designer model itself fell on 28% of trials, including four of five runs on the battery’s strongest single item, and replicate the central findings on a second ten-task battery designed, under a pre-registered protocol, by a model from the other lab (an internal replication: the same screening models and implementing agent, a different designer).

2 Related work

Template-override failures. GSM-Symbolic [2] shows numeric perturbation of familiar problems degrades all frontier models; MATH-Perturb [3] finds a failure mode in which the model “memorizes the problem-solving techniques from the training set and blindly applies them without judging whether the modified settings are still suitable”; the *unpuzzles* study [4] shows trivialized versions of famous puzzles score *below* the originals, with a context-shift control isolating surface similarity as the cause; Knights-and-Knaves role-flipping collapses accuracy to near zero [5]. Our instrument traps are the procedure-level analog: the memorized template is a *tool invocation* (“status is field 9”) rather than a solution schema.

Distribution-sensitivity of reasoning. McCoy et al. [6, 7] show accuracy tracks output probability and task frequency even in deterministic tasks, persisting in reasoning-optimized models. Token-bias studies [8] show logic-preserving lexical changes (Linda→Bob) shift outcomes. The named-idiom boundary we observe (§7.3) is consistent: traps whose defense is a high-frequency named procedure (strip whitespace, weight your means, stratify) were caught at 100% by the two development-phase screening configurations (§4).

Distraction and premise sensitivity. GSM-IC [9] and the GSM-Symbolic no-op clause show irrelevant-but-plausible context degrades solutions; premise-order effects exceed 30 points [10]. Inverse scaling in test-time compute [11] shows extended reasoning can amplify distractor pickup, motivating our effort 2×2 (§6.3).

Agentic failure modes. Coding agents show an *action bias*, proposing unnecessary changes on up to 65% of tasks where no change is required [12]; our false-premise trap was nonetheless caught by both screening configurations (§4), suggesting rapid saturation of that family. CoT-faithfulness results [13] motivated grading final artifacts rather than traces. The elicitation account of SimpleBench failures (that multi-step prompting recovers much of the lost performance [14]) is

directly tested in the agentic setting by our warned ablation (§6.4), which supports a mitigation-with-residue reading rather than a complete elicitation deficit.

Evaluation gaming. METR reported record rule-breaking by GPT-5.6 Sol (exploiting bugs in evaluation infrastructure and extracting hidden test cases) and judged its capability estimates non-robust as a result [15]. Our environments contain no grader and no hidden tests; in transcript audits for out-of-scope file access, Sol showed zero such access in 50 runs (§5) while falling at 82% (Table 2); a high rule-breaking rate in METR’s environment does not imply out-of-scope access in ours, though this does not by itself show that rule-breaking and trap-falling are separate capabilities.

3 Trap design: the five-condition frame

Iterative development (eight battery generations, §4) converged on five conditions that our successful traps satisfy and our failed traps each violate. We present the frame as *design heuristics with preliminary predictive use* (the later generations were designed to the conditions, with predictions registered before their runs) rather than as a validated taxonomy: conditions 3 and 5 are partly outcome-conditioned as stated, and preregistered validation with blinded raters on a novel battery remains future work (§8).

A trap aims to make the agent’s natural workflow produce a specific, plausible, wrong answer. It should be:

1. **Unnamed:** the defense must not be a retrievable named idiom. (One failed design hid a trailing space in a status value; every agent normalized whitespace reflexively before counting, without ever registering an anomaly; the defense is drilled, so the check runs without prompting.)
2. **Non-failing on habitual paths:** no crash, warning, or hard error on any common tool route. (One failed design serialized a window of timestamps as epoch seconds; the first strict `to_datetime` call raised, and the error message made the anomaly explicit; a trap that fails loudly converts itself into a bug report.)
3. **Not action-forcing under habitual checks:** heads/tails/samples, `describe()`, null counts, per-file ranges, and duplicate-ID checks may surface the anomaly but must not *compel* investigation. (Two failed designs shuffled rows so that two key formats appeared together in the first rows of any `head()`; every agent that saw the mixture investigated it; an anomaly landing in the habitual sample forces the question.)
4. **Plausible in output:** the wrong answer must be plausible on its face.
5. **Able to survive its own discovery:** if the anomaly surfaces, a benign story must be available (a zero-error window reads as a quiet stretch; “some disputes lack a merchant code,” verbatim from a run on the `nullmerch` task of Table 1).

Condition 5 is the most important for interpretation: in multiple documented transcripts, agents *surfaced* the anomaly with their own diagnostics and narrated it away.

4 What agents catch: screening and negative controls

Both halves of the suite were built by the same loop: a designer model (Claude Fable 5 for the first half, GPT-5.6 Sol for the second; §9) proposes candidate tasks; each candidate is screened by one

run each against GPT-5.5 and Claude Opus 4.8 (single runs per screener; screening, not statistics; all transcripts released); a candidate survives if and only if at least one screener falls; survivors enter the battery. The halves differ in evidential status: the Fable half’s survival criterion is de facto, characterized retrospectively from its development records, while the Sol half’s was pre-stated in a protocol registered before any design or run. What the screeners caught along the way is itself evidence, and it isolated the instrument-precondition family by elimination.

The Fable-designed half. Fable designed across eight generations, deriving the five-condition frame of §3 in the process; most candidate families were caught immediately by both screeners:

- *Documented tells*: a suggestively named leakage feature, a coding convention stated in a data dictionary, rotated log files visible in a directory listing; both models read them directly out of the artifacts before any computation.
- *Misleading authority*: a product manager’s note in the task folder endorsing a leaky feature: both models excluded the feature anyway, GPT-5.5 deriving the leak from deployment-time temporal reasoning before it had even opened the data dictionary.
- *False task premises*: a ticket confidently blaming an innocent currency module for a revenue shortfall: both models located the real cause (a silent row drop in ingestion), and Opus computed the counterfactual total under the accused fix to prove the module innocent.
- *Prior-confirming encodings*: an outcome flag whose naive reading reproduced a famous dataset’s survival statistics: both models derived the inversion from an insurance-ledger field definition and cross-checked it against a second document before computing.
- *Undocumented unit mixes*: one payment processor exporting integer cents beside another exporting dollars: caught by per-source profiling, in one case by aligning the two distributions at matched quantiles before summing.
- *Stale documentation*: a schema note asserting one row per order over row-grain that had migrated to line items: both models resolved the conflict toward the data and flagged the document.
- *Named statistical gotchas*: unweighted means over weighted data, float64 ID collisions, Simpson’s reversal from imbalanced arms, timezone-offset aggregation: caught in 8/8 model–task cells, each via the drilled idiom the gotcha is named for (transaction weighting, string-typed IDs, stratification with a confound diagnosis, local-time conversion).

The families that survived screening (parser-format windows, reader-default swallows, duplication, and their relatives) were refined into the battery of §5, and the failed designs were retained and run as negative controls under the battery protocol on the two screening configurations ($n=5$ each, hence /10). These behaved consistently with the frame: refunds and zeropad, whose mixed key formats were visible on inspection (condition 3), fell 0/10 each; status, whose trailing-space value was defeated by reflexive whitespace normalization (condition 1), fell 0/10 (the battery task `trailspace` survives that same reflex by hiding its space in a join key rather than a filtered value; §7.3); and the *documented* variants of two battery traps (restamp and netgross, with the hazard stated in the README; their undocumented twins are studied in §6.2) fell partway (3/10 and 2/10); a doc-cue protects only when read-and-acted-on. All attempted traps, including failures, are released.

The Sol-designed half. Sol worked behind an information firewall: it received the five-condition frame of §3 with every trap-identifying example redacted, plus the environment and generator conventions; it never saw the Fable-designed tasks, their mechanisms or results, or this manuscript; and from its second round it received the screening transcripts of its own candidates only, matching the Fable half’s development, where Fable, as both designer and operator, read all screening traces (the same agent operated the loop for both halves; §9). Its first round retraced the Fable half’s earliest failure: all eight candidates documented their own traps, and both screeners caught all eight by reading the documentation. With transcript feedback, its second and third rounds produced eleven survivors; the first ten were frozen (SHA-256 manifest, no post-hoc swaps) before any battery run. The registered protocol is released with its three logged amendments and an operator log; the released artifacts and ledgers label the halves *Battery One* and *Battery Two*.

5 Methods

The experiment, in outline: each agent configuration is pointed, in a fresh isolated session, at a folder containing a realistic but silently corrupted dataset and asked for a single business figure; the run’s final tagged answer is then scored against keys computed by the task generator as correct, planted (a fall), other-error, or no-answer. Twenty tasks in two ten-task batteries $\times$ ten configurations $\times$ five runs gives the 1,000 scored battery runs reported in §6. The rest of this section specifies the pieces.

Environments. Twenty synthetic task environments across the two batteries (Table 1; the Sol-designed half’s protocol: §4), each a small realistic folder (CSV, JSONL, and log files plus a plain README) under a neutral `/private/tmp` path, generated by seeded scripts with self-verifying assertions (the naive path must run without failing; the gap must be material; the planted and true answers must both be computed at generation). Contamination scans verify that no study vocabulary (task names, mechanism labels, answer values) appears anywhere in the deployed environment. Fresh pristine redeploys before every run.

Subjects. Ten configurations of eight models, run through each lab’s own shipped agent CLI. OpenAI: GPT-5.4-mini (**xhigh**), GPT-5.5 (at product effort **high** and **xhigh**), and GPT-5.6 Sol, Terra, and Luna, the flagship, mid, and fast tiers (each **xhigh**; `codex exec`, workspace-write OS sandbox). Anthropic: Claude Sonnet 5 (**high**), Claude Opus 4.8 (at **high**, the CLI default, and **xhigh**), Claude Fable 5 (**high**; `claude -p`, user-level configuration stripped, tool-scoped permissions). Isolation was verified per model by a probe run, a preliminary session in which the agent is asked to enumerate every instruction, memory, or configuration text visible in its context: no instruction files, no memory content, no hook injections. Comparisons are of **model–scaffold configurations as shipped**, not model weights (§8).

Protocol. $n=5$ fresh runs per task per configuration; identical neutral prompts requesting a specific figure with a machine-parseable final line; temperature not configurable in either CLI (“as-shipped” sampling). **A run falls if its final tagged answer matches the planted answer under the mechanical key**, the generator-computed answer key applied by script with pre-set numeric tolerances (released with the scorer); a wrong non-planted answer is an *other-error*, not a fall. Three runs across the two batteries produced no answer (two infrastructure failures unrelated to the models; one run that died mid-output): all are retained in the outcome tallies and excluded from rate denominators (997 of 1,000 answered). A pre-stated *caveat rule* was applied uniformly: a run that reports the planted value but explicitly flags it as a possible data artifact scores as a

catch, not a fall. The rule re-scored two runs on the Fable-designed half; five further candidate cases arose on the Sol-designed half, were reviewed under the rule, and none was re-scored (§6.6). The initial adjudicator was Fable, a disclosed conflict; every manual adjudication and exclusion was independently re-made from the raw transcripts by the human author (§9). Aggregate CIs are cluster-bootstrapped over tasks (runs within a task are not independent; the boundary interval [100%, 100%] for GPT-5.4-mini is the mechanical output of a 10-cluster bootstrap at the boundary and should not be read as certainty).

Out-of-scope-access scan. Both CLIs log every tool invocation, so a run’s transcript is the complete record of its filesystem access. All 1,000 battery transcripts, plus the Sol half’s design- and screening-phase transcripts, were scanned mechanically (pattern match over commands and outputs for research paths, key files, and scoring scripts), with every hit read manually. No run accessed answer keys, generators, or any research artifact; the sole match across the corpora was a benign common-noun use of “scratchpad” in one Fable-half run.

6 Results

6.1 Fall rates

Every rate below derives from two released *run ledgers*, one per half: machine-readable tables listing, for each run, its configuration, task, round, and scored outcome. Per-half rates use the corresponding ledger alone; combined rates pool both. (In the ledgers, GPT-5.5 configurations appear under their CLI-era label **Codex5.5**.) Scoring is by script against the generator-computed keys; the caveat adjudications and all exclusions are documented in the ledgers.

Across the combined twenty-task suite (1,000 runs), the ten configurations fall between 48% and 100%, and the family separation is strict within each battery and in combination: every OpenAI-family configuration falls above every Anthropic-family configuration on the Fable-designed half (82–100% vs. 26–41%), on the Sol-designed half (92–100% vs. 70–88%), and combined (90–100% vs. 48–62%), with pooled falls of 560/598 (OpenAI-family, 94%) vs. 220/399 (Anthropic-family, 55%). The nearest clustered intervals still overlap slightly (Sol combined [78%, 98%] vs. Opus-high [42%, 80%]).

Split by designer, the ordering held in both halves while the size of the gap did not: pooled falls are 89% (OpenAI-family) vs. 31% (Anthropic-family) on the Fable-designed half, a 58-point gap, against 98% vs. 79% for the same families on the Sol-designed half, a 19-point gap, so part of the Fable half’s gap magnitude was designer-flavored. The ordering itself replicates across the halves: the same two configurations are the most careful on both (Fable and Opus-xhigh), Opus 4.8’s effort scaling reproduces (41%→26% on the Fable half; 82%→70% on the Sol half), and the anomaly-narration signature recurs on both halves (§6.6). The direction of the family difference is therefore not a designer artifact, though its magnitude is designer-sensitive (registered protocol: §4; confounds: §8). The Fable-designed half carries a complementary credential: it was frozen the day before GPT-5.6’s public release, so the three 5.6 configurations were unavailable during its development and uninvolved in its selection, making their 82–90% on that half (Table 2) the study’s only prospective results.

One candidate explanation for the residual family difference is lab lineage (training and product culture around standing audit routines), but the design cannot rank it above the remaining alternatives: scaffold and product-policy differences, adaptive task-selection bias (both batteries were iterated against GPT-5.5 and Opus 4.8), and product-objective specialization. With only

two labs there is no replication at the lab level, and the effort 2×2 (§6.3) removes only the effort alternative; distinguishing the remainder requires additional labs and crossed scaffolds (§8).

Error concentration. Four-outcome scoring over all 1,000 runs: 212 correct, 780 planted, 5 other-error, 3 no-answer. **Of 785 answered-but-wrong runs, 780 (99.4%) produced the exact planted answer** (99.7% on the Fable-designed half, 99.1% on the Sol-designed half); planted-answer falls were 780/1,000 (78%) of all runs. (The Fable half’s sole other-error: a Sonnet 5 run that correctly rejected the trap, then named the wrong full day. The Sol half’s four include one answer exactly $10\times$ the planted value and two configurations independently converging on the same alternative wrong answer.) The selected traps thus produce highly concentrated, repeatable errors.

Per-task structure. (Full tables: Appendices A and B.) Two things stand out. First, saturation: **nullmerch** fell in 49/50 unwarned runs (sole escape: one Fable 5 run, §6.5), and the Sol-designed half contains *five* such tasks (fairview, iris, kepler, alder, birch, each 50/50) where eight generations of Fable design produced one; what discrimination that half retains concentrates in **aster** (28/50) and **bramble** (35/50), caught almost entirely by Fable and Opus-xhigh. Second, the transcripts show what the catching configurations did differently: on the two most family-separating tasks (NA-region and restamp-terse, 0–20% per Anthropic configuration vs. 80–100% per OpenAI configuration), the Anthropic configurations reconciled counts recomputed from the raw file against their parsed results and audited duplicates, and the OpenAI configurations did not. The parser-window class (drift/canary/jsonwin) pooled at 78% (116/149) with per-configuration rates from 13% to 100%, and **trailspace** was caught by every configuration except GPT-5.4-mini and the GPT-5.6 mid and fast tiers (§7.3).

6.2 The doc-hint matched pairs

Two traps were run in documented and undocumented versions (the documented version adds one README sentence naming the trap’s hazard) after the documented versions were diagnosed as condition-1 violations. The hinted and unhinted variants ran interleaved in the same batches on the same day, identical prompts and protocol, GPT-5.5 at xhigh and Opus at high in both; the README sentence is the designed difference, though the variants are distinct generated datasets rather than a single-file toggle (the unhinted variants entered the battery as **restamp-terse** and **netgross-quiet**). The comparison is therefore of two matched trap variants differing by the README hint:

The README hint was associated with a 40–60-point drop in GPT-5.5’s fall rate; its audits on these tasks are substantially externally cued. The hinted absolute rates are notable in their own right: with the hazard described in plain language in the README, GPT-5.5 still fell on 3/5 and 2/5 of runs: documentation that names the check elicited it only about half the time. Opus’s restamp defense survived de-hinting (self-generated); its netgross defense was partly hint-driven. This also yields a design constraint: documentation added to make a trap’s ground truth fair doubles as a cue that defuses it, so airtight traps must be *self-grounding*, their ground truth derivable from data invariants alone.

6.3 Effort 2×2

Crossing configuration family $\times$ product effort setting (GPT-5.5 at high/xhigh; Opus 4.8 at high/xhigh), the full Fable-designed half, $n=5$: GPT-5.5 is effort-invariant (44/50 at both settings, with the same per-task profile); Opus improves with effort (41% $\rightarrow$ 26%). The Sol-designed half

repeats the pattern (GPT-5.5 100% at both settings; Opus 82%→70%). The configuration gap persisted under both nominal settings and widened at matched-xhigh (88% vs. 26% on the Fable half). Because “high” and “xhigh” are product labels rather than cross-lab calibrated compute, these results remove the effort *setting* as an explanation, not effort per se. The inverse-scaling literature’s prediction that added reasoning could amplify distractor pickup (§2) did not materialize for Opus on either battery.

6.4 Warned ablation and its residues

A one-paragraph hygiene preamble (reconcile records matched vs. in scope; treat zeros, gaps, and surprises in intermediates as suspected bugs; verify format assumptions across the whole file; naming no trap; verbatim text in the released artifacts) was prepended, for each development-phase configuration, to every task that configuration fell on unwarned: nine of the ten Fable-designed tasks for GPT-5.5 (at xhigh; 27 runs) and six for Opus 4.8 (at high; 18 runs), $n=3$ per task. Runs with the preamble are *warned*; the original battery runs are *unwarned*. Because the two warned rates are computed over different task sets, they are not directly comparable to each other:

- GPT-5.5: 98% (44/45 unwarned on those nine tasks) → 22% (6/27) warned. Residues: burst **3/3** (checking whether event rows are distinct records never triggered, even under instruction), netgross-quiet 2/3, restamp-terse 1/3. Warned GPT-5.5 caught nullmerch 3/3.
- Opus 4.8: 69% (20/29 unwarned on those six tasks) → 17% (3/18) warned. Residue: nullmerch **3/3** (verbatim from one run: “Among *actual* merchant codes, RUBY leads”); the excluded group was seen, classified as not-real-codes, and dropped under an instruction that explicitly says to “account for every record that was dropped.”

Explicit instruction does not close the hole. The preamble helps (it transfers beyond the failures it was written against, fixing traps its text never mentions, and reduces fall rates roughly fourfold), but under a direct instruction to perform exactly the missing checks, each model still failed outright on at least one task (GPT-5.5 on all three burst runs; Opus on all three nullmerch runs), with partial residues on two more. Whatever this deficit is, it is not fully repaired by telling the agent what to do. These results indicate a prompt-sensitive verification policy; cleanly separating elicitation from capability would additionally require length-matched-placebo and equal-compute controls (§8). The instruction also carries a real cost: warned runs used a median of $2.5\times$ the tokens of unwarned runs on the same tasks (per-task range $1.2\text{--}4.2\times$; measured on the `codex-CLI` side only, as the Claude CLI does not report usage).

6.5 Designer as subject

Claude Fable 5, the model that designed every trap in the Fable half, was run cold as a subject (fresh sessions, probe-verified isolation, no memory of the design work): **14/50 (28%)**, inside the Anthropic band. Its single nullmerch catch (1/5) is the only unwarned escape from that task observed, and its transcript shows *preemptive instrument distrust*: predicting pandas’ NULL-swallow and the exact trap answer before loading the file (`keep_default_na=False`), then verifying NULL is a genuine merchant. The same weights fell 5/5 for their own cross-column swap trap (netgross-quiet) and 3/5 for their own duplicated-burst trap (burst). Fable’s aggregate rate lies within the observed Anthropic-configuration range, providing no affirmative evidence of a large designer advantage; this is compatible with, but does not establish, a knowledge-to-policy explanation (design knowledge carried into the model’s standing verification habits). Sol, run cold as a designer-subject on the

Sol-designed half under the same isolation, fell 48/49 on its own traps, showing none of Fable’s partial self-immunity: designer self-recognition is not a general property of the designer role.

6.6 Qualitative signature: anomaly narration

Across configurations and tasks, the deepest documented failure is **narration**: agents surface the anomaly with their own diagnostics, then absorb it with a benign story. Opus, having produced an hourly table with a 4.5-hour zero-error hole (09:30–14:00) on a service with a ~ 29 errors/hour baseline, reported it flatly (“Hours 10:00–13:00 logged zero errors”) without investigation. GPT-5.5, in a development-phase screening run, printed `value_counts(dropna=False)` showing NaN as the largest region and wrote “keeping missing values separate from named billing regions.” On partial-day, GPT-5.5 printed the anomalous `as_of_ts` 09:15 value in its own final output in three of five runs and answered without comment. The Sol-designed half reproduces the signature in exactly the runs where a caveat catch would live: five of its falls end in caveat-like wording (mentions of anomalies, exclusions, or double-counting), and every one uses that wording to *deny* the artifact rather than flag it. One Opus run cites the README note about paired controllers and concludes it “introduce[s] no double-counting” while double-counting. Reviewed under the pre-stated caveat rule (§5), none of the five converts to a catch. The traces establish omission-then-plausible-report; whether plausibility *causes* the omission is a hypothesis (§7.1).

7 Discussion

7.1 Candidate mechanisms

Agents in these configurations omit coverage validation after silent coverage-changing transformations and report plausible, task-satisfying answers, at rates that vary sharply by configuration (Table 2). Candidate drivers that our design cannot separate: satisficing stopping rules (a plausible requested figure exists; further checks have low perceived value); observability (the transformation removes the evidence needed to detect the discrepancy, so noticing requires a second, independent measurement); salience allocation (docs, crashes, large null groups, and famous gotchas attract scrutiny; quiet discrepancies do not); and cost-calibrated verification (full reconciliation carries the token cost noted in §6.4 and risks false alarms, so the omission may be a miscalibrated tradeoff rather than an absent capability).

7.2 A two-parameter picture of agent auditing

The doc-hint pairs, warned residues, and effort 2×2 (all Fable-half experiments; the Sol half’s effort pattern matches, §6.3) suggest a two-parameter description: **where checks come from** (externally cued vs. self-generated) and **whether effort scales them**. GPT-5.5: externally cued (a single README sentence separates hinted from unhinted fall rates by 40–60 points), effort-flat. Opus 4.8: self-generated (survives de-hinting), effort-scaled (41%→26% on the Fable half, 82%→70% on the Sol half), but threshold-dependent: a small-enough anomaly with a benign story defeats it at every setting tested, warned included.

7.3 The named-idiom boundary

In development, every trap whose defense is a *named, drilled* procedure (strip whitespace, weight your means, read big IDs as strings, stratify before concluding, convert timezones) was caught at 100% by both screening configurations (§4). The traps that survived lack names for their required

check (“your grep can go blind mid-file”). Namedness is, however, confounded with cue loudness, training frequency, and task structure (dose-response designs are needed to separate them), and the boundary is configuration-bounded rather than absolute: `trailspace`’s defense is arguably nameable (normalize keys before joining), yet it trapped GPT-5.4-mini and the GPT-5.6 mid and fast tiers. The named-idiom boundary is therefore a design heuristic for the configurations tested, not a law. For benchmark design, the implication is that published failure modes should be expected to saturate.

7.4 Relation to the single-turn motif

The single-turn literature’s motif recurs here at the procedure level (template retrieval, an unheeded invalidating detail, systematic convergence, and partial elicitation fixes), and the named-idiom boundary (§7.3) suggests the *defenses* rest on the same distributional statistics: agents are protected exactly where the required check is frequent enough to have a name. The parallel is a similarity of pattern, not evidence of a shared cause. It is closest for the narration failures, where the anomaly was in the agent’s context and was explained away (the same shape as SimpleBench’s in-prompt misdirection), and weaker for pure coverage omissions, where the invalidating evidence was never sampled at all, a situation SimpleBench questions never create. We offer the parallel as an organizing hypothesis for the dose-response work in §8.

7.5 Practical defenses

Four standing rules address the mechanisms behind most observed failures (not all: the warned residues of §6.4 resisted direct instruction): (1) after any filter or parse, reconcile rows-in-scope vs. rows-matched and explain the residual; (2) treat any zero or step-change in a bucketed intermediate as a suspected bug, not a fact; (3) check that co-reported aggregates from one table are mutually consistent; (4) before reusing a format assumption, sample the *middle* of the data. These cost tokens (warned runs used a median of $2.5\times$ the unwarned spend, §6.4); whether and when that spend is worth it remains open (§8).

8 Limitations and future work

Limitations. (1) Scaffold $\neq$ model: all comparisons are of shipped agent products; lab, weights, CLI, system prompts, sandboxing, and product objectives are bundled. (2) Adaptive development: both batteries were iterated against the same two evaluated configurations and selected under a fools-at-least-one-screener rule, so all fall rates are conditional on that selection; only the Fable-designed half’s GPT-5.6 results are prospective, and even those may inherit lineage-shared selection effects. (3) The five-condition frame is partially outcome-conditioned as used; the Sol-designed half is a first predictive use by a second designer (§4), but blinded-rater validation is not yet done. (4) Effort labels are not cross-lab calibrated; sampling parameters are product defaults. (5) Designer lineage, tested: with a GPT-lineage designer the cross-lab *ranking* replicated with strict separation, so the direction of the gap is not a designer artifact; its *magnitude* shrank from 58 to 19 points. One battery per designer means designer is confounded with task set, chronology, and development history, so these are transfer results across the two halves, not estimates of a designer effect. Residual conflicts: both halves were implemented and operated by an Anthropic-lineage research agent (§9), and both designers saw both screeners’ transcripts during iteration, leaving untested the strong form of the designer-blind-spot hypothesis: that a designer can trap the other lineage, or fail to, from priors alone, without behavioral feedback. (6) Twenty tasks across two batteries, mostly single

instance per trap shape, two labs. (7) No human practitioner baseline: “an observant analyst catches these” rests on construction and the designer, not measurement. (8) Two protocol incidents (a prompt-contamination leak caught by a subject model reporting anomalous input, with the affected batch quarantined and rerun; a runner failure requiring one make-up wave) are documented in the released supplementary report (§9).

Future work. Preregistered taxonomy validation (observable condition definitions, blinded raters, a frozen novel battery including failed designs); placebo-preamble and equal-compute ablation arms, plus a self-description-preamble arm; an independently designed battery and crossed scaffolds (same models through the same harness); dose-response manipulation of anomaly size, placement, and plausibility within fixed tasks; hierarchical analysis over tasks, models, and labs; hygiene-tax quantification; a human practitioner baseline.

9 AI-contribution disclosure and reproducibility

This study was designed, implemented, and drafted by Claude Fable 5 (Anthropic) operating as a research agent under the direction of the human author, who set questions, scope, and methodological standards. Half the tasks were designed by GPT-5.6 Sol (OpenAI) under the pre-registered protocol described in §4, with the same Anthropic-lineage agent acting as harness operator only (brief authoring, mechanical execution, screening, scoring); the designer identities are material to interpreting §6.5, §4, and limitation (5). Subject models were run as shipped products with no fine-tuning. One design-phase incident is disclosed in the released operator log: asked to *study* screening transcripts, the designer instead executed a task prompt embedded in one of them, itself an instance of the failure family under study.

All artifacts are released at <https://github.com/AlexanderJulianKing/silent-precondition-traps> and permanently archived at <https://doi.org/10.5281/zenodo.21383254>: generators (seeded), environments, prompts, per-run transcripts (~1,200 scored runs: the 1,000 battery runs plus warned, doc-hint, and negative-control runs, with probes and quarantined runs included alongside their incident documentation), answer keys, scoring scripts, the run ledgers from which every figure in this paper derives, adjudication decisions, exclusions, failed and discarded trap designs from both designers, the supplementary research report with the full development history, and the Sol half’s pre-registered protocol with its logged amendments, operator log, and complete design-session transcripts. All reported numbers derive mechanically from the released ledgers and scoring scripts. The human author independently re-adjudicated every manual-adjudication case in both batteries (all confirmed), and verified a stratified sample of the Fable half’s ledger, all quotations, and the answer-extraction scripts against raw transcripts (the verification checklist and quote-audit scripts are released).

A Per-task falls, Fable-designed half (from the released ledger)

Denominators below 5 reflect the two no-answer runs (Opus@high drift r1; Terra NA-region r3). Pooled descriptive class rates: parser-window (drift, canary, jsonwin) 116/149 (78%); reader-swallow (NA-region, nullmerch) 77/99 (78%); duplication (burst, restamp-terse) 62/100 (62%).

References

- [1] Philip and Hemang. *SimpleBench: The Text Benchmark in which Unspecialized Human Performance Exceeds that of Current Frontier Models*. Technical report, Oct. 2024. <https://simple-bench.com>
- [2] I. Mirzadeh et al. *GSM-Symbolic*. arXiv:2410.05229.
- [3] K. Huang et al. *MATH-Perturb*. arXiv:2502.06453.
- [4] A. Malek et al. *Frontier LLMs Still Struggle with Simple Reasoning Tasks (unpuzzles)*. arXiv:2507.07313.
- [5] C. Xie et al. *On Memorization of Large Language Models in Logical Reasoning*. arXiv:2410.23123.
- [6] R. T. McCoy et al. *Embers of Autoregression*. arXiv:2309.13638.
- [7] R. T. McCoy et al. *When a Language Model is Optimized for Reasoning...* arXiv:2410.01792.
- [8] B. Jiang et al. *A Peek into Token Bias*. arXiv:2406.11050.
- [9] F. Shi et al. *Large Language Models Can Be Easily Distracted by Irrelevant Context*. arXiv:2302.00093.
- [10] X. Chen et al. *Premise Order Matters*. arXiv:2402.08939.
- [11] A. P. Gema et al. *Inverse Scaling in Test-Time Compute*. arXiv:2507.14417.
- [12] *Coding Agents Don't Know When to Act*. arXiv:2605.07769.
- [13] Y. Chen et al. *Reasoning Models Don't Always Say What They Think*. arXiv:2505.05410.
- [14] S. Sane and A. McLean. *A NotSo Simple Way to Beat Simple Bench*. arXiv:2412.12173.
- [15] METR. *Summary of METR's predeployment evaluation of GPT-5.6 Sol*. June 2026. <https://metr.org/blog/2026-06-26-gpt-5-6-sol/>

Additional grounding literature cited in the supplementary review: Sprague et al. arXiv:2409.12183; Nezhurina et al. arXiv:2406.02061; McKenzie et al. arXiv:2306.09479; Zhang et al. arXiv:2405.00332; Wu et al. arXiv:2307.02477; Madaan and Yazdanbakhsh arXiv:2209.07686.

B Per-task falls, Sol-designed half (from the released ledger)

Table 1: The two frozen batteries: tasks 1–10 designed by Claude Fable 5, tasks 11–20 by GPT-5.6 Sol (design protocol: §4). Mechanism labels are descriptive groupings.

#	Task	Silent violation	Naive → true answer
<i>Fable-designed half</i>			
1	drift	mid-file <code>up=</code> token shifts status field (a configuration change, later reverted)	1,407 → 10,014 5xx; wrong worst endpoint
2	canary	log window switches to logfmt (<code>ERROR→level=error</code>)	1,365 → 6,659 errors; wrong peak hour
3	jsonwin	log window switches to JSON lines	1,198 → 5,888 errors; wrong worst module
4	NA-region	region code "NA" in pandas default <code>na_values</code>	EMEA → NA as top region (38% of rows become NaN)
5	nullmerch	merchant short-code literally NULL (13% of rows)	RUBY → NULL as top merchant
6	burst	collector re-emitted a 10-min window 8× (same event IDs)	42,312 → 40,000 events
7	restamp-terse	replayed window with <i>fresh</i> event IDs (defeats <code>nunique</code>)	40,404 → 40,000 events
8	netgross-quiet	net/gross columns swapped for a 4-day window	\$1.057M → \$0.985M net revenue
9	partial-day	final day’s rollup snapshotted mid-morning (<code>as_of_ts</code>)	Jun 30 → Jun 13 as lowest-revenue day
10	trailspace	payment refs carry trailing space post-migration	40% → 83% invoices paid
<i>Sol-designed half</i>			
11	aster	one late-month handheld build records kilometres inside a mile column	227,561.6 → 211,580.6 mi
12	bramble	carrier-accepted replacement labels create unique records for one physical parcel	17,950 → 17,000 parcels
13	cobalt	telemetry partition renews session IDs at midnight	82.2 → 101.8 min average session
14	fairview	pre-disposition disconnects are absent as rows but consume switch-sequence positions	82.62% → 78.59% answer rate
15	iris	double-door units emit two synchronized controller histories	83.1 → 74.4 occupied units
16	kepler	checkout currency is presentation metadata; amounts are already USD	\$1.091M → \$1.026M GMV
17	alder	opaque upstream channels are additive near-copies of counted downstream channels	734,760.5 → 606,804.3 kWh
18	birch	service field attached one parcel late; weight and charge stay on the correct row	14,604.9 → 12,229.3 kg rush weight
19	cinder	firmware heartbeat approvals mixed into customer approvals	\$0.797M → \$0.761M card sales
20	dogwood	the instructor’s ordinary badge is recorded alongside customers at every class meeting	16,620 → 15,132 attendances

Table 2: Fall rates by configuration on the twenty-task suite, split by designer and combined. Cells: falls (rate) [clustered 95% CI]; $n=5$ per task per cell; **higher is worse**. Denominators exclude the three no-answer runs (§5): one Terra and one Opus-high run on the Fable-designed half, one GPT-5.6 Sol run on the Sol-designed half. GPT/OpenAI configurations ran through the `codex` CLI, Claude/Anthropic configurations through Claude Code.

Configuration	Fable-designed	Sol-designed	Combined
GPT-5.4-mini (xhigh)	50/50 (100%) [100, 100]	50/50 (100%) [100, 100]	100/100 (100%) [100, 100]
GPT-5.5 (high)	44/50 (88%) [68, 100]	50/50 (100%) [100, 100]	94/100 (94%) [83, 100]
GPT-5.5 (xhigh)	44/50 (88%) [68, 100]	50/50 (100%) [100, 100]	94/100 (94%) [83, 100]
GPT-5.6 Luna (xhigh)	45/50 (90%) [74, 100]	49/50 (98%) [94, 100]	94/100 (94%) [85, 100]
GPT-5.6 Sol (xhigh)	41/50 (82%) [60, 98]	48/49 (98%) [94, 100]	89/99 (90%) [78, 98]
GPT-5.6 Terra (xhigh)	43/49 (88%) [73, 100]	46/50 (92%) [80, 100]	89/99 (90%) [81, 97]
Claude Opus 4.8 (high)	20/49 (41%) [18, 65]	41/50 (82%) [60, 100]	61/99 (62%) [42, 80]
Claude Sonnet 5 (high)	15/50 (30%) [10, 54]	44/50 (88%) [68, 100]	59/100 (59%) [39, 78]
Claude Fable 5 (high)	14/50 (28%) [8, 52]	38/50 (76%) [50, 100]	52/100 (52%) [32, 72]
Claude Opus 4.8 (xhigh)	13/50 (26%) [6, 48]	35/50 (70%) [42, 96]	48/100 (48%) [28, 68]

Trap	GPT-5.5 falls (hinted $\rightarrow$ unhinted)	Opus 4.8 falls
restamp	3/5 $\rightarrow$ 5/5	0/5 $\rightarrow$ 0/5
netgross	2/5 $\rightarrow$ 5/5	0/5 $\rightarrow$ 3/5

Task	mini	Luna	Terra	Cdx@xh	Cdx@h	Sol	Opus@h	Sonnet	Fable	Opus@xh
drift	5/5	5/5	5/5	4/5	4/5	4/5	4/4	1/5	0/5	3/5
canary	5/5	5/5	5/5	5/5	5/5	5/5	4/5	2/5	2/5	3/5
jsonwin	5/5	5/5	5/5	5/5	5/5	5/5	3/5	5/5	0/5	2/5
NA-region	5/5	4/5	3/4	5/5	5/5	5/5	0/5	1/5	0/5	0/5
nullmerch	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	4/5	5/5
burst	5/5	5/5	5/5	5/5	5/5	5/5	0/5	0/5	3/5	0/5
restamp-terse	5/5	5/5	5/5	5/5	5/5	4/5	0/5	0/5	0/5	0/5
netgross-quiet	5/5	5/5	5/5	5/5	5/5	5/5	3/5	0/5	5/5	0/5
partial-day	5/5	5/5	2/5	5/5	5/5	3/5	1/5	1/5	0/5	0/5
trailspace	5/5	1/5	3/5	0/5	0/5	0/5	0/5	0/5	0/5	0/5

Task	mini	Luna	Terra	Cdx@xh	Cdx@h	Sol	Opus@h	Sonnet	Fable	Opus@xh
aster	5/5	4/5	5/5	5/5	5/5	4/5	0/5	0/5	0/5	0/5
bramble	5/5	5/5	3/5	5/5	5/5	5/5	2/5	5/5	0/5	0/5
cobalt	5/5	5/5	5/5	5/5	5/5	4/4	4/5	4/5	5/5	4/5
fairview	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
iris	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
kepler	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
alder	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
birch	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
cinder	5/5	5/5	3/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
dogwood	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5	3/5	1/5

Table 3: Battery Two falls per task (falls/answered, $n=5$ per cell; **cobalt**-Sol denominator is 4 after one API-capacity no-answer). Five tasks fell 50/50 across all configurations; cross-lab discrimination concentrates in **aster** and **bramble**. Higher is worse.